CSC 316

Handout: SVG — How a Chart Is Drawn

What this is. The bridge from data to pixels. Once you can read SVG, a chart stops being opaque — it's a handful of shapes positioned by numbers. **Companion to Lab 3** (useful from Lab 1 on, central once you reach encoding).

> **What's live now.** From **Lab 1**, read **§1–3** as reference. **§4** and the appendix use Svelte (`{#each}` , `$props` , `$derived`) and switch on with **Labs 2–3** — skip them until then.

Examples use the **earthquake scatter** from Lab 3: depth on x, magnitude on y, one circle per quake.

---

1. The SVG coordinate system

An `<svg>` is a drawing canvas. You give it a width and height, and everything inside is positioned by numbers relative to its **top-left corner**.

```
<svg width="600" height="400"> … </svg>
```

That corner is `(0, 0)`. And here is the fact that trips up every single person once:

> **y grows *downward*.** `y = 0` is the **top** edge. `y = 400` is the **bottom**. On a chart, "higher value" therefore means a **smaller** y number.

This is why a y-scale in every chart you'll read looks inverted:

```
const yScale = d3.scaleLinear()
  .domain([6, 8.8])         // data: magnitude 6.0 to 8.8 (the catalogue is M6.0+)
  .range([height, 0]);      // pixels: BOTTOM to TOP  ← the flip
```

Read that range as *"the bottom of the chart represents magnitude 6.0; the top represents 8.8."* Write `.range([0, height])` by mistake and your chart is upside down — the biggest earthquakes sit along the bottom. It's the most common encoding bug in the course, and now you can spot it in one glance.

`viewBox` and responsiveness. You'll often see:

```
<svg viewBox="0 0 600 400" width="100%"> … </svg>
```

`viewBox="0 0 600 400"` declares an internal coordinate system 600 wide and 400 tall; `width="100%"` lets the element stretch to its container. The drawing scales to fit, so all your internal numbers stay simple while the chart resizes. Coordinates inside SVG are **unitless** — they're viewBox units, not pixels or millimeters.

---

2. The core shapes

Five elements draw the charts in this course.

```
<rect  x="10" y="20" width="40" height="120" />   <!-- x,y = TOP-LEFT corner -->
<circle cx="150" cy="80" r="6" />                 <!-- cx,cy = CENTER -->
<line  x1="0" y1="200" x2="600" y2="200" />       <!-- two endpoints -->
<path  d="M0,100 L50,80 L100,120" />              <!-- a sequence of moves -->
<text  x="300" y="390">depth (km)</text>          <!-- x,y = text BASELINE -->
```

Two of those positioning rules differ: a `<rect>` is placed by its **top-left**, but a `<circle>` by its **center**. So a bar's `y` is where its *top* goes — which is why bar code computes `y={yScale(d.value)}` and `height={yScale(0) - yScale(d.value)}` rather than just a height.

`<path>` is the general one. Its `d` attribute is a mini-language: `M` = move to, `L` = line to, `C` = curve, `Z` = close. You will almost never write one by hand — D3's line and area generators produce the string for you — but recognizing `d="M…L…"` as "a line through these points" is enough.

`<g>` groups. A `<g>` has no appearance; it exists to hold other elements and move them together with `transform`:

```
<g transform="translate(50, 20)">
  <!-- everything in here is shifted 50 right and 20 down -->
</g>
```

This is the **margin convention** in one line: wrap the plot in a `<g>` translated by the left and top margins, and every coordinate inside can be measured from the plot's own corner, leaving room for axes and labels outside it. `transform` also does `rotate(-90)` — how you get a vertical y-axis label.

Order is z-order. SVG has no z-index: elements drawn later sit on top. If your gridlines are covering your data, they're simply after it in the markup.

---

3. Stroke vs. fill

Every shape has two colorable parts, and mixing them up is a classic invisible-chart bug.

- **`fill`** — the interior
- **`stroke`** — the outline

```
<circle cx="60" cy="60" r="20" fill="steelblue" stroke="white" stroke-width="1" />
<line x1="0" y1="0" x2="100" y2="0" stroke="#333" />   <!-- a line has NO interior -->
```

A `<line>` or an unclosed `<path>` has no interior, so **`fill` does nothing on it** — set `stroke` or it's invisible. Conversely a `<circle>` with only a `stroke` is a hollow ring.

The vocabulary for telling two series apart:

Attribute	Effect
`stroke-width`	Line thickness
`stroke-dasharray="4 2"`	Dashes — 4 units on, 2 off. **The colorblind-safe way to distinguish lines.**
`stroke-linecap` / `-linejoin`	How ends and corners are shaped
`fill-opacity` / `stroke-opacity`	Transparency of each part separately
`opacity`	Transparency of the whole element

> **Opacity is how you survive overplotting.** 53,940 diamonds or 4,777 listings drawn at full opacity is a solid mass. `fill-opacity="0.15"` turns the pile-up into visible density — where marks stack, the color deepens. It's the cheapest fix for a crowded scatter, and it's why `opacity` appears in nearly every scatter prompt in the labs.

Styling from CSS works too, and is often cleaner:

```
<circle class="quake" cx={…} cy={…} r="4" />

<style>
  .quake { fill: steelblue; fill-opacity: 0.6; }
  .quake:hover { fill: orange; }
</style>
```

Note it's `fill`, not `color` — see The Browser, HTML & CSS §2.

---

4. From data to marks

Here's the mental model that makes generated chart code legible. **A chart is a list of shapes whose attributes are computed from data.** That's all.

Chart	Is really	One row becomes
Bar chart	a list of `<rect>`s	one bar
Scatterplot	a list of `<circle>`s	one dot
Line chart	**one** `<path>`	one *vertex* — the whole series is a single element
Labels	a list of `<text>`s	one label

That line-chart row is worth pausing on: a scatter of 155 quakes is 155 elements, but a line through 155 points is **one** `<path>` whose `d` string encodes them all. It's why you can't hover an individual point on a plain line chart without adding invisible circles on top.

In Svelte, the binding is an `{#each}`:

```
{#each quakes as d}
  <circle
    cx={xScale(d.depth)}      <!-- data value -> pixel, via the scale -->
    cy={yScale(d.mag)}
    r="4"
    fill="steelblue"
    fill-opacity="0.6"
  />
{/each}
```

Read it as a sentence: *for each quake, draw a circle whose horizontal position is its depth run through the x-scale, and whose vertical position is its magnitude run through the y-scale.*

Those two expressions are the encoding. `cx={xScale(d.depth)}` is the exact line where a number in a CSV becomes a position on screen — the data-to-pixels moment. When a lab asks you to point at the encoding in your code, it's pointing at this.

Everything else follows the same shape. Want size to carry magnitude? `r={rScale(d.mag)}`. Color to carry depth? `fill={colorScale(d.depth)}`. **Each visual channel is one attribute set from one scale of one column** — and adding a channel means

adding an attribute, which is exactly how a chart quietly becomes overloaded.

Appendix — Reading a generated chart

A complete small scatter, annotated. This is the shape of what Claude will hand you.

```
<script>
  import * as d3 from 'd3';
  let { quakes } = $props();

  // The margin convention: an outer box, and an inner plot area
  const width = 600, height = 400;
  const margin = { top: 20, right: 20, bottom: 45, left: 55 };
  const innerW = width  - margin.left - margin.right;    // 525
  const innerH = height - margin.top  - margin.bottom;   // 335

  // Scales map DATA -> PIXELS (see the D3 handout)
  let xScale = $derived(
    d3.scaleLinear().domain([0, d3.max(quakes, d => d.depth)]).range([0, innerW])
  );
  let yScale = $derived(
    d3.scaleLinear().domain([6, 8.8]).range([innerH, 0])   // ← the y-flip
  );
</script>

<svg viewBox="0 0 {width} {height}" width="100%">

  <!-- ONE <g> shifted by the margins: everything inside is measured
       from the plot's own top-left, leaving room for the axes outside -->
  <g transform="translate({margin.left},{margin.top})">

    <!-- the marks: one <circle> per row -->
    {#each quakes as d}
      <circle
        cx={xScale(d.depth)}
        cy={yScale(d.mag)}
        r="4"
        fill="steelblue"
        fill-opacity="0.6"
      />
    {/each}

    <!-- the x-axis line, pushed to the BOTTOM of the plot area -->
    <line x1="0" y1={innerH} x2={innerW} y2={innerH} stroke="#333" />

    <!-- axis label. Centered horizontally; y is BELOW the plot (larger = lower) -->
    <text x={innerW / 2} y={innerH + 38} text-anchor="middle">depth (km)</text>

    <!-- y-axis label, rotated upright -->
    <text
      transform="rotate(-90)"
      x={-innerH / 2} y={-40}
      text-anchor="middle"
    >magnitude</text>

  </g>
</svg>
```

Four things to be able to point at, in any chart code:

1. **The `{#each}` —** where one row becomes one mark.
2. **A `scale(d.field)` call —** where a number becomes a pixel.
3. **The `range([innerH, 0])` —** the y-flip, and the reason “up” is a smaller number.
4. **The `<g transform="translate(…)">` —** the margin convention making room for axes.

If you can name those four in a chart you didn’t write, you can read any chart in this course.